

Guía manejo de archivos

Concepto Principal: La línea de comandos permite examinar y procesar archivos de texto plano sin necesidad de abrirlos en un editor interactivo (como `nano` o `vim`). El comando `cat` (concatenate) imprime el contenido de uno o varios archivos directamente en la salida estándar de la terminal. Por lo general, se reserva para visualizar archivos cortos. Para archivos comprimidos, existe `zcat`, que permite leer el contenido sin tener que descomprimir el archivo previamente en el disco.

Ejemplos prácticos:

~ `cat /etc/passwd`: Muestra todo el contenido del archivo de contraseñas del sistema.

~ `cat /etc/passwd /etc/group`: Lee ambos archivos secuencialmente y los une en la misma salida por pantalla.

- Perspectiva desde la ciberseguridad: Utilizar `cat` es la forma más segura de revisar un archivo de configuración crítico. Al no abrir un editor, eliminas el riesgo de modificar accidentalmente el archivo o cambiar sus metadatos (como la fecha de modificación), lo cual es vital en auditorías.

2. Paginadores (`more` y `less`)

Concepto Principal: Cuando los archivos son muy extensos para usar `cat`, recurrimos a los paginadores, que dividen el contenido en pantallas o "páginas". Al igual que con `cat`, existen `zmore` y `zless` para paginar directamente archivos comprimidos.

Característica	<code>more</code>	<code>less</code>
Navegación	Restrita, muestra el porcentaje avanzado.	Avanzada, permite moverse libremente.
Finalización	Termina automáticamente al llegar al final.	Requiere presionar <code>q</code> para salir.
Búsqueda	Limitada.	Permite buscar patrones, resaltar resultados y saltar entre ocurrencias.
Uso en el sistema	Herramienta clásica.	Es el paginador por defecto del comando <code>man</code> .

Comandos clave de navegación en `less`:

- ~ `/criterio`: Busca el patrón indicado dentro del texto.
- ~ `f` y `b`: Avanza (`f` orward) o retrocede (`b` ackward) una página.
- ~ `g` y `G`: Salta directo al principio (`g`) o al final (`G`) del archivo.
- ~ `q`: interrumpe el proceso y sale del paginador.

Ejemplo practico:

- ~ `less /etc/passwd`: Abre el archivo de manera paginada.

3. Extremos de Archivos (`head` y `tail`)

Concepto Principal: Estas herramientas están diseñadas para mostrar exclusivamente las primeras (`head`) o las últimas (`tail`) líneas de un archivo. Si no se especifica otra cosa, ambos comandos devuelven exactamente 10 líneas por defecto.

Comandos clave y ejemplos prácticos:

~ `head -n 20 /var/log/syslog`: Muestra las primeras 20 líneas del archivo en lugar de las 10 predeterminadas.

~ `tail -n 50 /var/log/messages`: Imprime las últimas 50 líneas desde el final del archivo

~ `tail -f /var/log/auth.log`: Mantiene el archivo abierto y muestra en tiempo real los datos que se van agregando al final.

- **Perspectiva de Ciberseguridad:** El parámetro `-f` (follow) de `tail` es fundamental para el monitoreo en vivo. Te permite ver ataques de fuerza bruta por SSH en `/var/log/auth.log` o revisar bloqueos de un firewall en tiempo real mientras ocurren.

4. Estadísticas y Diferencias (`wc` y `diff`)

Concepto Principal: El comando `wc` cuenta la cantidad de líneas, palabras y caracteres (bytes) dentro de un texto. Por su parte, `diff` es una herramienta de comparación que examina dos archivos y expone sus diferencias exactas, línea por línea.

Comandos clave y ejemplos prácticos:

~ `wc -l /etc/passwd`: Devuelve únicamente la cantidad total de líneas del archivo.

~ `wc -c /etc/passwd`: Cuenta la cantidad de bytes.

~ `cat /etc/passwd | wc -l`: Al usar pipes (tuberías), suministramos la salida de `cat` a `wc`. En este caso, solo nos devuelve el número, omitiendo el nombre del archivo en la salida.

~ `diff test1 test2`: Compara ambos archivos. Señala con `<` las líneas del primer archivo y con `>` las del segundo donde se detectan las diferencias.

- Perspectiva de Ciberseguridad: `diff` es tu mejor amigo para la auditoría de configuraciones. Te permite verificar exactamente qué cambió en un archivo de configuración crítico (como `sshd_config` o reglas de `iptables`) después de una actualización o una sospecha de intrusión.

5. Búsqueda de Archivos (`locate`, `find`, `whereis`, `which`)

Concepto Principal: Como administrador, frecuentemente necesitarás ubicar archivos en el sistema. Existen distintas herramientas dependiendo de la precisión y velocidad requerida.

Comando	Método de Búsqueda	Casos de Uso
<code>locate</code>	Lee desde una base de datos interna, es casi instantáneo.	Búsquedas rápidas y generales. Requiere que la base se actualice con <code>updatedb</code> .
<code>find</code>	Recorre la estructura real del sistema de archivos en vivo.	Búsquedas precisas, combinando criterios granulares (propietario, permisos, fecha, tamaño).
<code>whereis</code>	Busca binarios, manuales (man pages) y código fuente.	Cuando necesitas saber dónde está instalado un programa y su documentación.
<code>which</code>	Busca únicamente ejecutables dentro de la variable de entorno <code>\$PATH</code> .	Para confirmar qué versión de un comando se ejecutará al tipearlo.

Comandos clave y ejemplos prácticos:

~ `locate -i config`: Busca cualquier archivo que contenga "config", ignorando mayúsculas y minúsculas.

~ `find /etc -name '*.conf'`: Busca dentro de `/etc` todos los archivos que terminen en `.conf`. El asterisco se encierra en comillas para que la shell no lo expanda

~ `find /etc -type f -user root -mtime 60`: Busca archivos regulares (`-type f`), que pertenezcan a `root`, y modificados hace exactamente 60 días.

~ `find / -type f -perm 777 -exec chmod 644 {} +`: Ejecuta una acción sobre los resultados. Busca archivos con permisos totales (777) y les aplica `chmod 644`. El `{}` representa a los archivos encontrados.

~ `whereis -b top`: Devuelve únicamente la ruta del archivo binario del comando `top`.

~ `which ls`: Indica la ruta del ejecutable (ej. `/bin/ls`).

Perspectiva de Ciberseguridad: El comando `find` es una herramienta clave en el *Hardening* (securización) y respuesta a incidentes. Buscar archivos vacíos (`-empty`), archivos con permisos inseguros como `777`, o archivos ejecutables modificados recientemente por usuarios no autorizados, es parte de la rutina diaria de un analista de seguridad.

6. Filtrado y Búsqueda de Texto (**grep**)

Concepto Principal: **grep** (Globally Regular Expressions Pattern) busca patrones o expresiones regulares dentro de uno o varios archivos y devuelve la línea completa donde encuentra la coincidencia. Es, sin lugar a dudas, la herramienta de filtrado más potente de la terminal.

Comandos Clave y Ejemplos Prácticos:

- ~ **grep -i lucca /etc/passwd**: Busca la palabra "lucca" ignorando la diferencia entre mayúsculas y minúsculas.
- ~ **grep -r linux /usr/share/doc**: Realiza una búsqueda recursiva dentro de todos los archivos de un directorio.
- ~ **grep -v error /var/log/syslog**: Invierte la búsqueda, devolviendo todas las líneas que no contengan la palabra "error".
- ~ **grep -c root /etc/group**: En lugar de mostrar las líneas, devuelve un número indicando la cantidad de veces que aparece la palabra "root".
- ~ **grep -n log /var/log/messages**: Imprime el número de la línea al lado del resultado encontrado.

Perspectiva de Ciberseguridad: Es la base del análisis de logs forense. Con **grep** podés extraer direcciones IP de atacantes, buscar códigos de error específicos (como un error HTTP 403 o 500 en logs de Apache), o identificar firmas de malware dentro de archivos sospechosos.

Por último, para reforzar el conocimiento realizamos:

Reto de Laboratorio (Lab 1)

Para tener algo de práctica, se propone este desafío simulado para resolver. Imaginá que estás auditando un servidor y necesitas hacer lo siguiente en una sola secuencia de análisis (no importa si se usa tuberías o comandos separados):

1. Buscá en todo el sistema (/) cualquier archivo de texto regular que tenga permisos 777.
2. De esos resultados, imaginá que encontraste un archivo sospechoso (puede ser un .log). Vas a leer solo sus últimas 15 líneas.
3. De esas 15 líneas, necesitás filtrar y contar cuántas veces aparece la palabra "failed" (ignorando mayúsculas y minúsculas).

Guia de solucion

1. Usamos el comando `find`, indicamos su ruta y con `-perm` especificamos el permiso.

```
root@UTN-Lucka:~# cd
root@UTN-Lucka:~# find / -type f -perm 777
/tmp/registros_accesos.log
```

Vemos que en `/tmp` se encuentra un `.log`

2. Para ver sus últimas 15 líneas usamos `tail` con el `-n 15` para indicarle la cantidad de líneas que deseamos ver, además indicamos la ruta.

```
root@UTN-Lucka:~# tail -n 15 /tmp/registro_accesos.log
Access granted
Failed login attempt
Access granted
failed password for root
Failed login
Access granted
Session opened
failed to authenticate
Connection closed
Access granted
Failed to start
Failed login
Access granted
Session closed
failed password
root@UTN-Lucka:~#
```


3. Para filtrar cuantas veces aparece la palabra “failed” usamos **grep** acompañado de **-i** (ignora mayúsculas y minúsculas) y **-c** (devuelve número indicando la cantidad), seguido de la palabra que queremos encontrar con la ruta del archivo.

```
root@UTN-Lucka:~# grep -c -i failed /tmp/registro_accesos.log
7
```

Sin embargo el paso 2 y 3 se puede resumir uniendo los dos comandos usando | (pipes), esto hace que el segundo comando tome la salida del primero.

```
root@UTN-Lucka:~# tail -n 15 /tmp/registro_accesos.log | grep -i -c failed
7
```

Por último, un cheat sheet para tener a mano

